

Méthodes d'optimisation à gradient

Théodore Cherrière, MCF CentraleSupélec, laboratoire GeePs
theodore.cherriere@centralesupelec.fr

Objectifs du cours

Compétences et savoir-faire

1. Définir un modèle (rappel)

- Formalisme : état, variable de conception, paramètres, critère
- Calcul de dérivées, méthode de l'adjoint

2. Evaluer l'état d'un modèle non-linéaire

- Méthode du point fixe
- Méthode de Newton

3. Mettre en œuvre une démarche d'optimisation non-contraainte

- Descente de gradient
- Méthode de Newton

4. Mettre en œuvre une démarche d'optimisation sous contrainte

- Pénalisation
- Lagrangien augmenté

wooclap



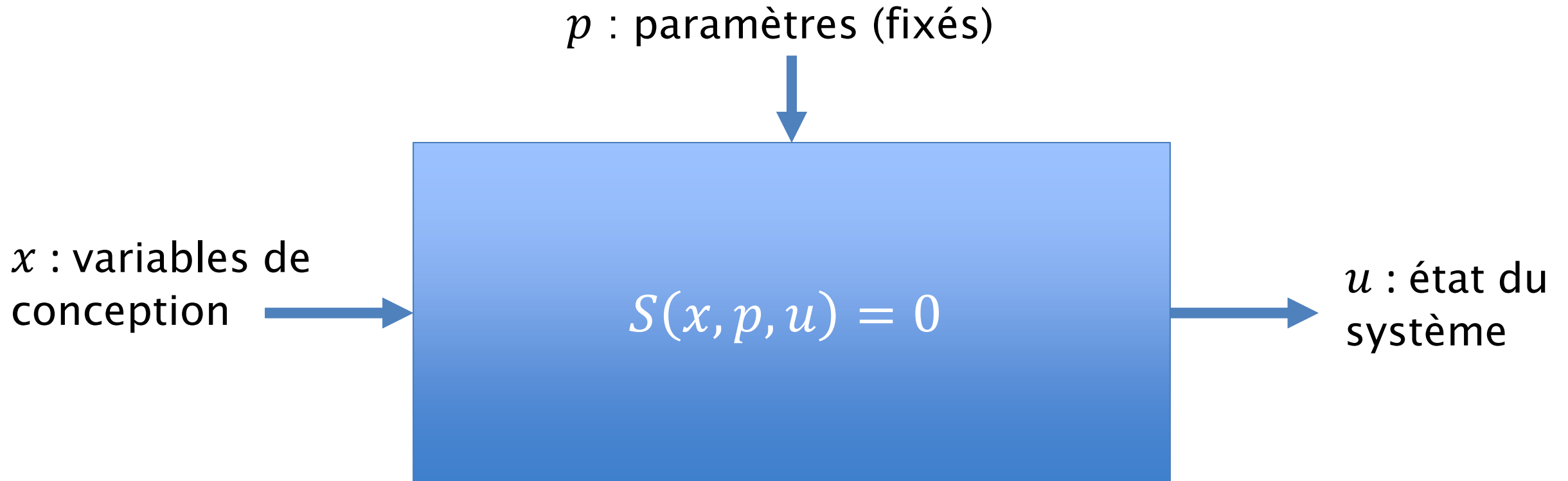
<https://app.wooclap.com/IQMOEJ>

I) DÉFINIR UN MODÈLE

Formalisme, vocabulaire, dérivation

Formalisme

Qu'entend-on par « modèle » ?

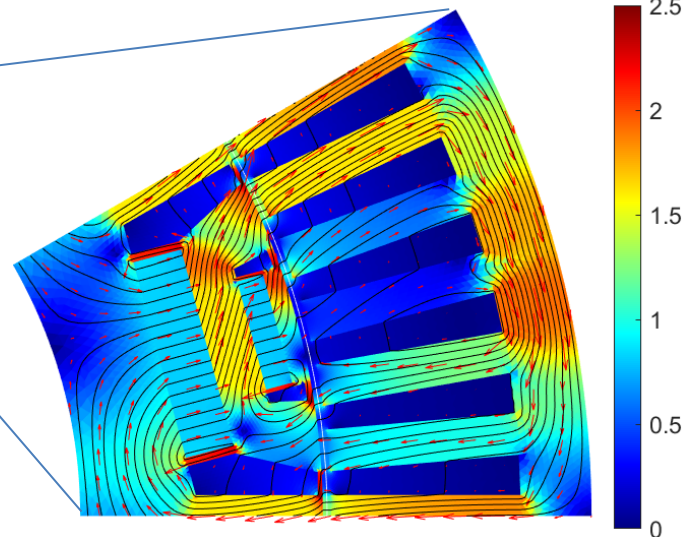


Formalisme

Exemple : simulation magnétostatique

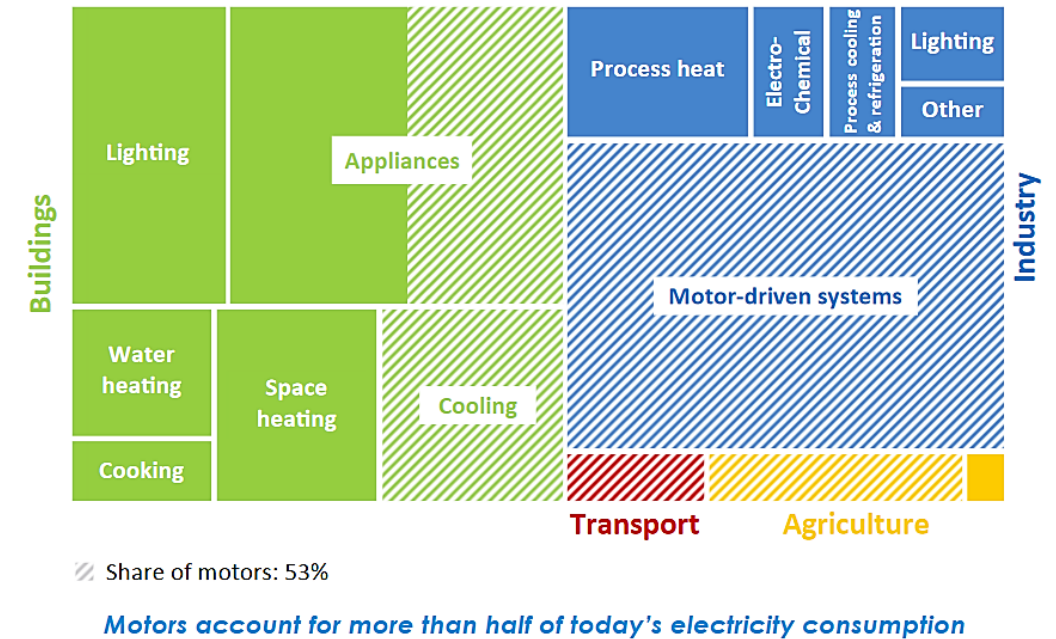
- *Moteur électrique*

Part dans la consommation mondiale d'électricité: + de 50% !



Induction magnétique (T)

Figure 7.9 ▷ Global total final electricity consumption by end-uses, 2014



Source: IEA analysis.

Quels sont les paramètres, variables de conception, équations du modèle ?

Formalisme

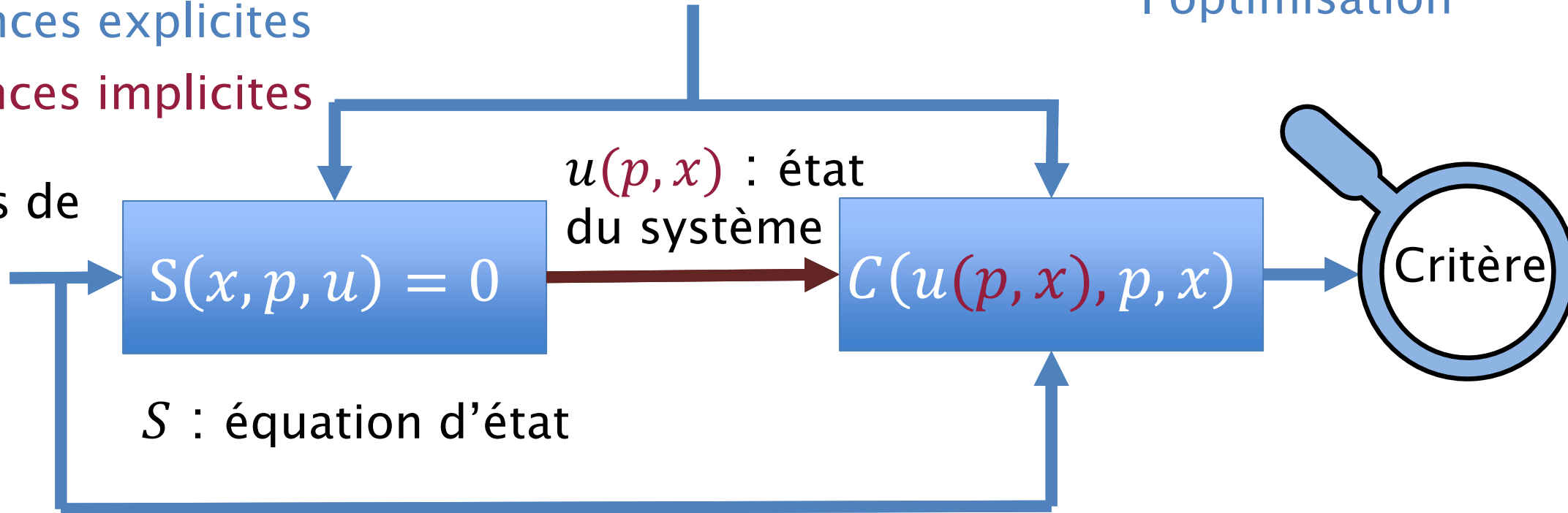
Critère

- Dépendances explicites
- Dépendances implicites

p : paramètres fixés

Grandeur d'intérêt pour l'optimisation

x : variables de conception



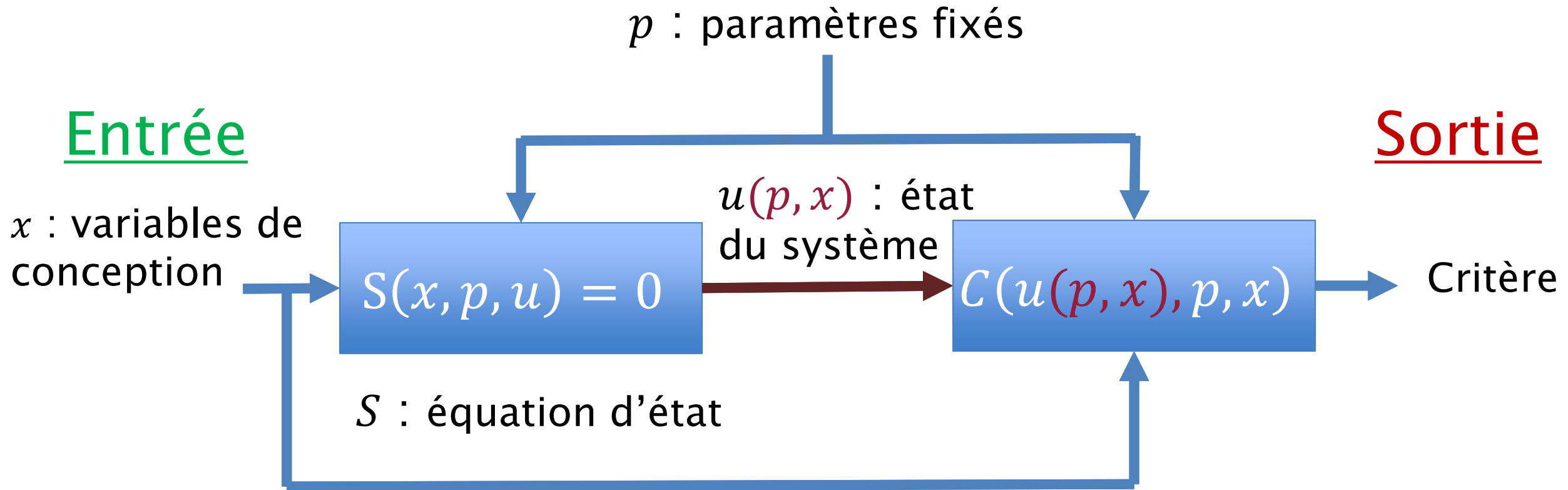
Les **grandeurs d'intérêts** dépendent généralement **explicitement** de **l'état** mais **implicitement** des **variables de conception**.

Problème direct

Définition

Entrée

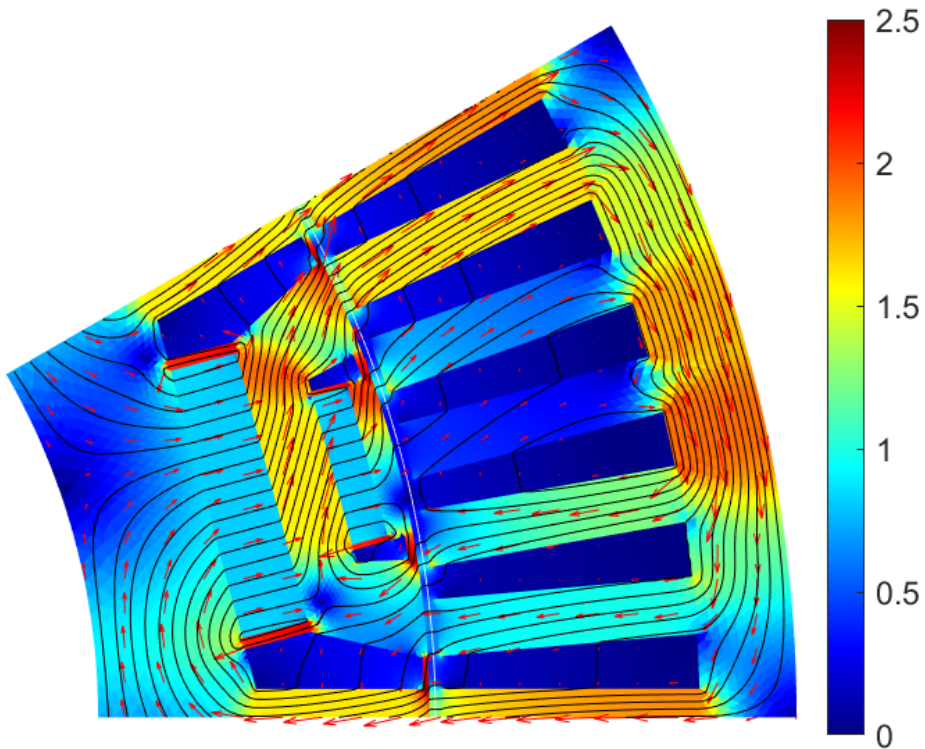
Sortie



Résoudre un problème direct est « facile », mais n'est que rarement une fin en soi.

Problème direct

Exemple : calcul de couple magnétique



Etat $u \rightarrow \vec{B}$
(induction magnétique)

Formule à partir de la répartition de $\vec{B}$ dans l'entrefer e (Arkkio, 1987) :

$$\rightarrow C(\vec{B}) = \frac{L_z}{\mu_0 \Delta R} \int_e r B_r B_\theta \, dS$$

Si on change **l'alimentation électrique** (p) ou la **géométrie** (x), l'état $\vec{B}$ (u) va changer, et donc $C(\vec{B}(x, p))$ également.

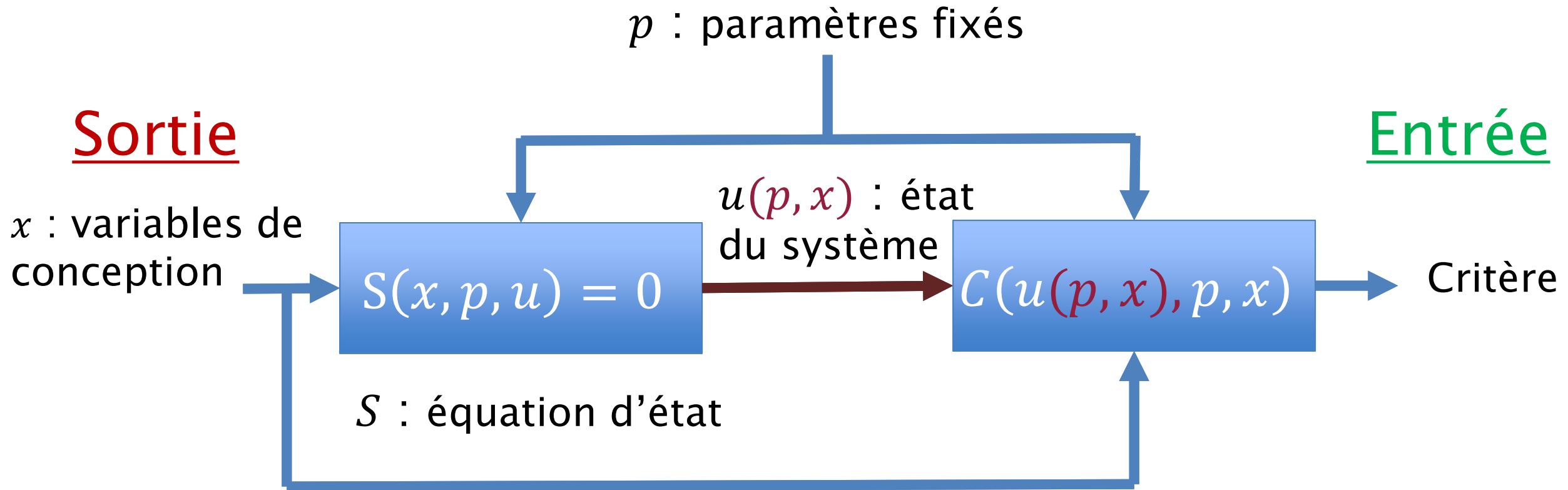
$\Rightarrow$ Dépendance **implicite** via $S(x, p, u) = 0$

Problème inverse

Définition

Sortie

Entrée



Le client s'intéresse généralement à un **problème inverse** : il recherche **les variables de conception** associées à des **performances attendues**.

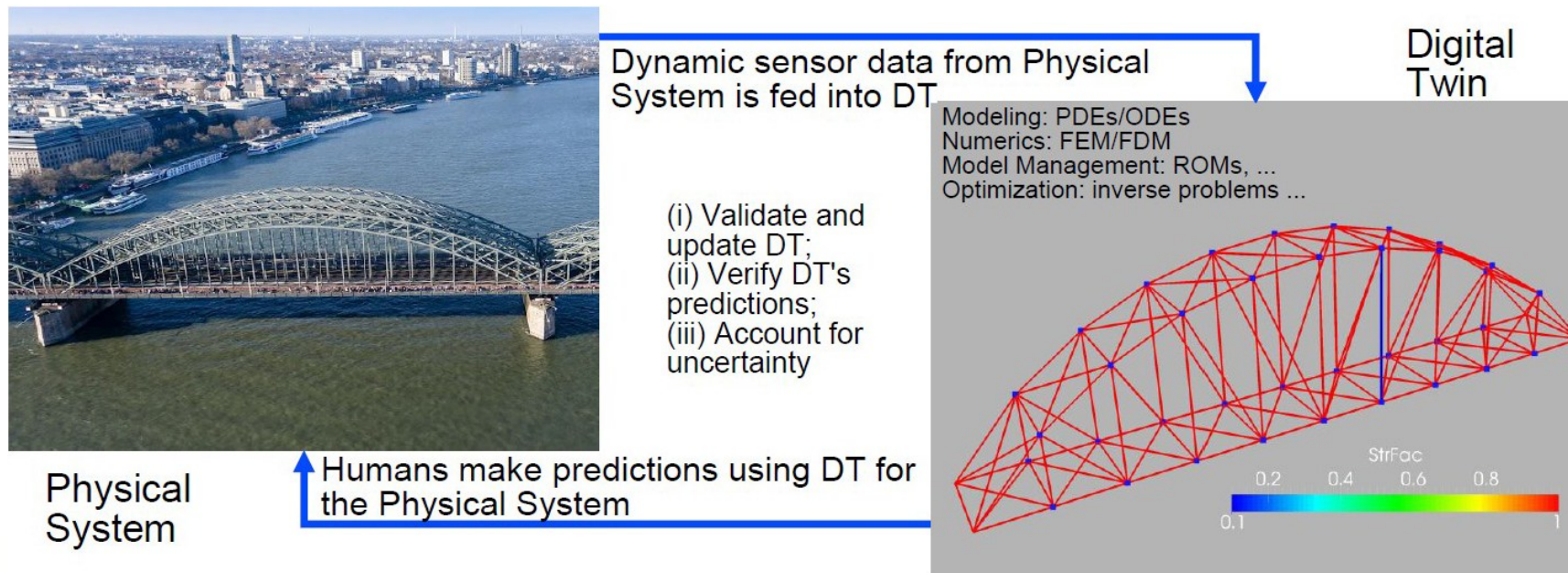
Problème inverse

Difficultés liées aux problèmes inverses

Résoudre un problème inverse est généralement difficile :

- Pas forcément existence, unicité ou stabilité de la solution x (problème **mal posé**) → nécessité de **régularisation**, de **contraintes**

Exemple: maintenance prédictive par jumeau numérique



Calcul de dérivées

Dans cette séance de cours, on s'intéressera à des méthodes:

1. de **résolution** de $S(x, p, u) = 0$;
 2. d'**optimisation** vis-à-vis de la variable de conception x ,
- qui sont basées sur des **dérivées premières** (plus efficace).

Remarque : x, p, u peuvent être scalaires, vectoriels, voire de dimension infinie.

Calcul de dérivées

Difficultés

- Première difficulté : **dimension et nature** des quantités à dériver

Ce que vous savez sûrement déjà dériver :

- $f: \mathbb{R} \rightarrow \mathbb{R}$
- $f: \mathbb{R}^n \rightarrow \mathbb{R}$
- $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$



Ce que vous ne savez peut-être pas encore dériver...

- $f: L^2(\Omega) \rightarrow \mathbb{R}$, avec $L^2(\Omega) = \{v: \Omega \rightarrow \mathbb{R}, |\int_{\Omega} v^2| < \infty\}$ et $\dim L^2(\Omega) = \infty$
- $f: \Omega \rightarrow \mathbb{R}$, avec Ω une *répartition de matière non paramétrée*

Dans ce cours, on supposera qu'on *sait* dériver, et on notera la dérivée de f

- f' si f ne dépend que d'une variable (éventuellement de dimension ∞)
- $d_x f$ la dérivée (**totale**) par rapport à x
- $\partial_x f$ la dérivée **partielle** par rapport à x si f possède plusieurs variables

Calcul de dérivées

Difficultés

- Deuxième difficulté : **dépendances implicites** entre variables.

on pose $y = 2x$, $f_1 = x + y$, $f_2 = 3x$

1) f_1 et f_2 sont-elles égales ?

Cela dépend de quoi on parle !

- **Quantités** : $f_1 = x + 2x = 3x = f_2$
- **Fonctions** : $f_1: x, y \mapsto x + y$
 $\neq f_2: x, y \mapsto 3x$

Calcul de dérivées

Difficultés

- Deuxième difficulté : **dépendances implicites** entre variables.

on pose $y = 2x$, $f_1 = x + y$, $f_2 = 3x$

2) Que valent $\partial_x f_1$ et $\partial_x f_2$?

• **Dérivée partielle** = « dériver en maintenant les autres variables constantes »

Utile si on considère des **fonctions**
 $\partial_x f_1 = 1$, $\partial_x f_2 = 3$

Calcul de dérivées

Difficultés

- Deuxième difficulté : **dépendances implicites** entre variables.

on pose $y = 2x$, $f_1 = x + y$, $f_2 = 3x$

3) Que valent $d_x f_1$ et $d_x f_2$?

•
Dérivée « totale »: règle de la chaîne $d_x f(x, y) = \partial_x f + \partial_y f d_x y$

Même quantité = mêmes dérivées « totales »

Calcul de dérivées

Estimation par différences finies

Si variable de conception x de dimension finie :

Différence avant $\Delta_{x_i} f = f(u(x_1, x_2, \dots, x_i + h, \dots, x_n)) - f(u(x_1, x_2, \dots, x_i, \dots, x_n))$

Différence centrale $\delta_{x_i} f = f\left(u\left(x_1, x_2, \dots, x_i + \frac{h}{2}, \dots, x_n\right)\right) - f\left(u\left(x_1, x_2, \dots, x_i - \frac{h}{2}, \dots, x_n\right)\right)$

$x \mapsto f(u(x))$ de classe $\mathcal{C}^1$	$x \mapsto f(u(x))$ de classe $\mathcal{C}^2$
$d_{x_i} f - \frac{\Delta_{x_i} f}{h} = O(h)$	$d_{x_i} f - \frac{\delta_{x_i} f}{h} = O(h^2)$

[Jupyter Notebook](#)

Calcul de dérivées

Pas complexe

Si variable de conception x de dimension finie et f est analytique :

Pas complexe $\underline{\Delta_{x_i} f} = f(u(x_1, x_2, \dots, x_i + \textcolor{red}{i}h, \dots, x_n)) - f(u(x_1, x_2, \dots, x_i, \dots, x_n))$

$x \mapsto f(u(x))$ de classe $\mathcal{C}^1$	$x \mapsto f(u(x))$ de classe $\mathcal{C}^2$	$x \mapsto f(u(x))$ de classe $\mathcal{C}^2$ analytique
$d_{x_i} f - \frac{\Delta_{x_i} f}{h} = O(h)$	$d_{x_i} f - \frac{\delta_{x_i} f}{h} = O(h^2)$	$d_{x_i} f - \frac{\text{Im}(\Delta_{x_i} f)}{h} = O(h^2)$ Pas d'erreur de calcul flottants

Simple à implémenter, robuste, fonctionne même si $f(u(x))$ boîte noire...
...mais doit être **répété** pour chaque composante de x !! **LENT** 

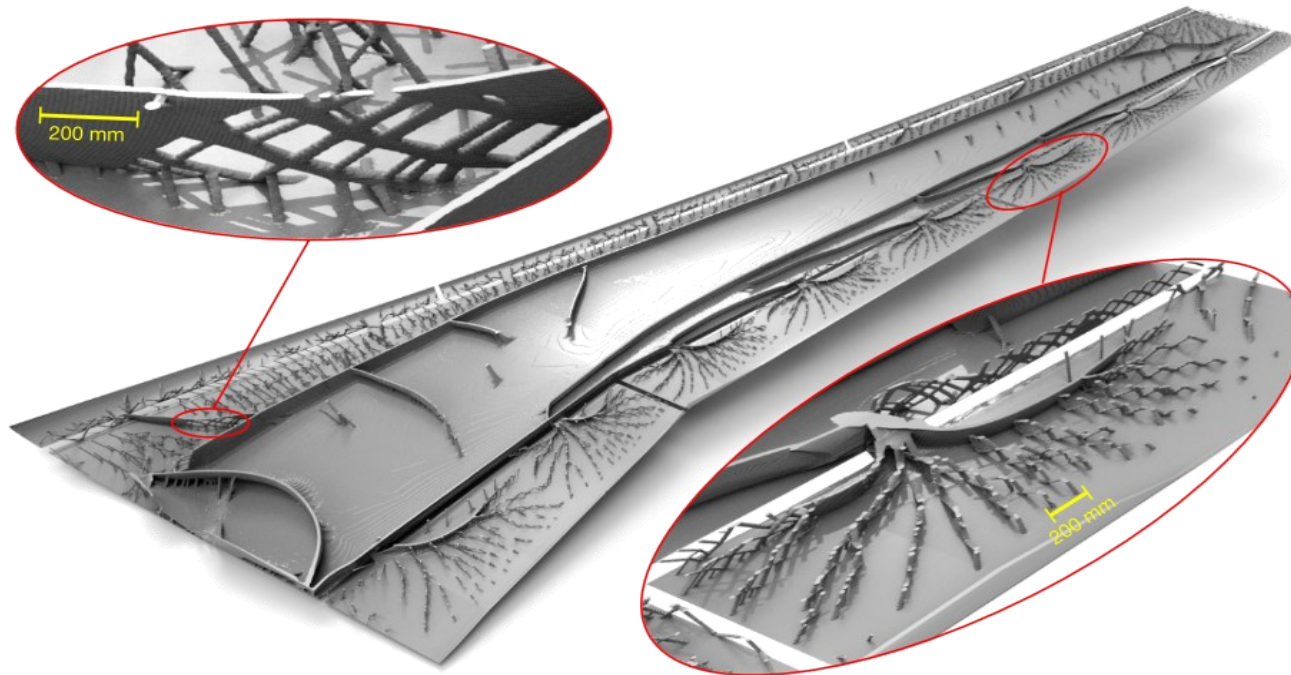
Calcul de dérivées

Problème de grande dimension

Giga-voxel computational morphogenesis for structural design

| NATURE | VOL 550 | 5 OCTOBER 2017

Niels Aage^{1,2}, Erik Andreassen¹, Boyan S. Lazarov¹ & Ole Sigmund¹



element i . The sensitivities of the objective function are determined efficiently without additional analysis using the **adjoint method**, and the filtering operation is included by the **chain rule**²². The optimization problem is solved using a nested approach and an optimality criteria algorithm^{2,23} (simple MATLAB code available



Figure 2 | Morphogenesis evolution, 3D printing and comparison to the hornbill bird beak. a, Steps of the morphogenesis process for the full-wing

Calcul de dérivées

Méthode de l'adjoint (principe formel)

On veut calculer $d_{x_i}f$. **Règle de la chaîne :**

$$d_{x_i}f = \partial_{x_i}f + \partial_u f d_{x_i}u \quad (1)$$

Étape 1 : dériver l'équation d'état pour faire apparaître $d_{x_i}u$ par la **règle de la chaîne** :

$$\begin{aligned} d_{x_i}S &= \partial_{x_i}S + \partial_u S d_{x_i}u = 0 \\ \Rightarrow \partial_u S d_{x_i}u &= -\partial_{x_i}S \quad (2) \end{aligned}$$

Étape 2 : introduire $\partial_u S$ dans (1)

$$\partial_u f d_{x_i}u = \partial_u f \partial_u S^{-1} \partial_u S d_{x_i}u = -\partial_u f \partial_u S^{-1} \partial_{x_i}S$$

Étape 3 : résoudre le système adjoint (***toujours linéaire***), indépendant de x_i

$$p \partial_u S = -\partial_u f \Leftrightarrow \partial_u S^* p = -\partial_u f$$

Finalement,

$$\boxed{d_{x_i}f = \partial_{x_i}f + p \partial_{x_i}S}$$

Calcul de dérivées

Méthode de l'adjoint

En résumé:

1. Calcul de l'état **physique** u (pas forcément linéaire = **long**); on résout
$$S(u, x) = 0$$

2. Calcul de l'état **adjoint** p (toujours **linéaire**, parfois **auto-adjoint** $\Rightarrow$ **rapide**) ;
on résout

$$\partial_u S^* p = -\partial_u f$$

3. On applique la formule suivante pour chaque composante x_i (**rapide**):

$$d_{x_i} f = \partial_{x_i} f + p \partial_{x_i} S$$

Calcul de dérivées

Méthode de l'adjoint

Exemple de problème à grande dimension
Optimisation topologique d'un transformateur

[Jupyter Notebook](#)

Calcul de dérivées

Dérivation symbolique et automatique

Autres approches:

- **Dérivée symbolique** (construction explicite d'une expression sur laquelle on applique les règles de dérivation usuelle)
- **Dérivée automatique** : les dérivées sont calculées numériquement et récursivement, basée sur la règle de la chaîne. On distingue:
 - L'accumulation directe (forward) : plus efficace quand la dimension de l'espace d'arrivée est grande
 - L'accumulation inverse (backward) : plus efficace quand la dimension de l'espace de départ est grand (ex : réseaux de neurones)
 - **Bibliothèques Python** : JAX, PyTorch, etc.

5-10 minutes de pause

Questions?

2) EVALUER L'ETAT D'UN MODELE

Méthode du point fixe, méthode de Newton

Méthode du point fixe

Objectif

On veut maintenant trouver u^* qui résout l'équation d'état

$$S(u^*, x, p) = 0$$

Comme x et p sont fixés, on écrit simplement

$$S(u^*) = 0$$

On recherche donc un algorithme qui génère une suite (u_k) , telle que

$$\lim_{k \rightarrow \infty} u_k = u^*$$

Méthode du point fixe

Principe

On pose $F: u \mapsto u + A(u) S(u)$, avec A un opérateur inversible quelconque. Résoudre $S(u^*) = 0$ équivaut à trouver un **point fixe** de F :

$$u^* = F(u^*)$$

Appliquons l'algorithme le plus simple possible :

$$u_{k+1} = F(u_k)$$

Et étudions sa convergence.

Méthode du point fixe

q - convergence

Supposons que u^* existe et que F est dérivable ; on considère u_k *proche* de u^* .
Formule de Taylor-Young :

$$\frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u_k) - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u^*) + F'(u^*)(u_k - u^*) + O(u_k - u^*)^2 - u^*\|}{\|u_k - u^*\|}$$

$$\Rightarrow \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} \rightarrow \|F'(u^*)\|$$

$\ F'(u^*)\ = 0$	$\ F'(u^*)\ \in]0,1[$	$\ F'(u^*)\ > 1$
Convergence superlinéaire	Convergence linéaire	Pas de convergence

Rapide

Le choix de A peut accélérer
ou stabiliser l'algorithme

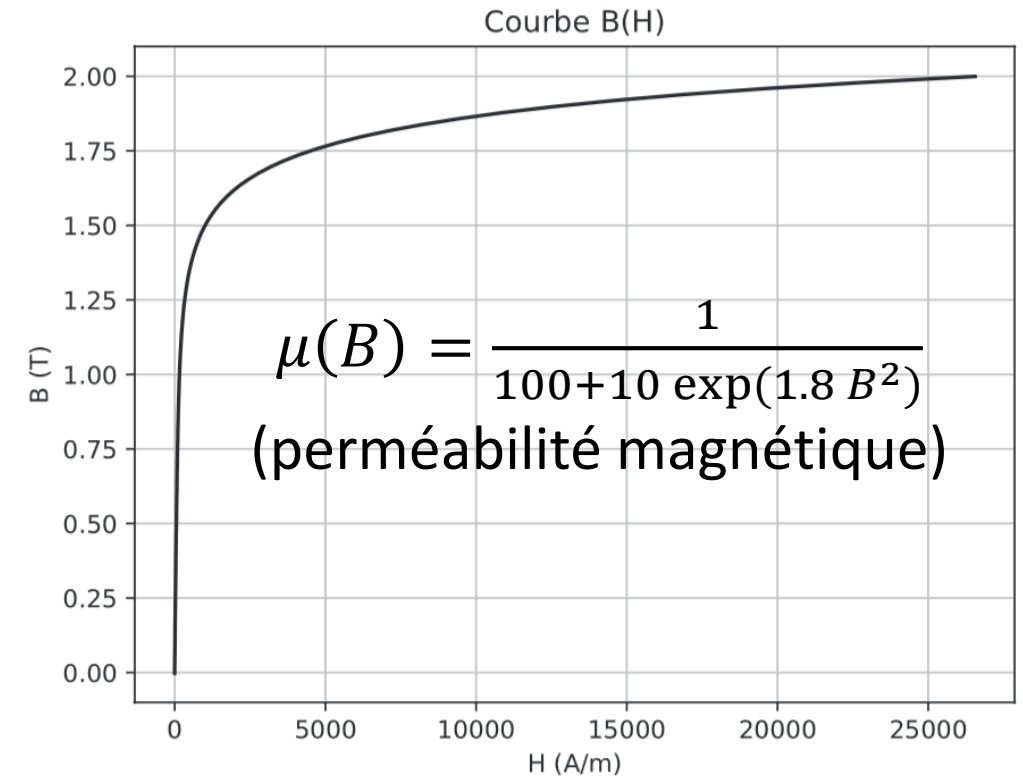
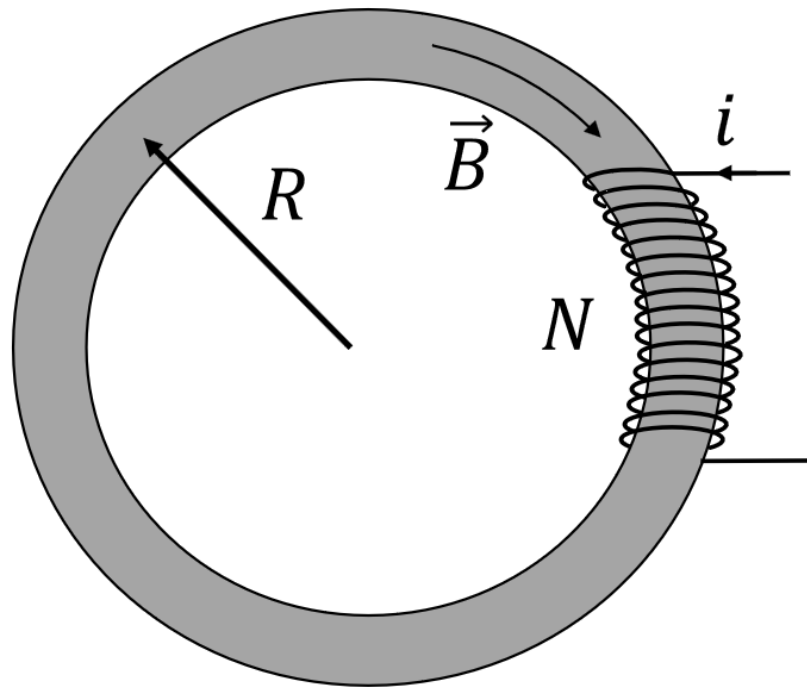
Instable

Méthode du point fixe

Exemple numérique

Induction magnétique dans un tore ferromagnétique

- $N = 100$
- $i = 5 \text{ A}$
- $R = 5 \text{ cm}$



[Livescript Matlab](#)

Méthode de Newton

Principe

- On veut résoudre $S(u) = 0$, on sait résoudre par point fixe $F(u) = u$ avec $F: u \mapsto u + A(u) S(u)$; écrivons la dérivée à la solution u^*

$$F'(u^*) = Id + \underbrace{A'(u^*) S(u^*)}_0 + A(u^*) S'(u^*) = Id + A(u^*) S'(u^*)$$

- Condition nécessaire de q-convergence superlinéaire :

$$\|F'(u^*)\| = 0 \Rightarrow A(u^*) = -(S'(u^*))^{-1}$$



Impossible si $S'(u^*) = 0$!

- On en déduit la **méthode de Newton** (Raphson) :

$$u_{k+1} = u_k - (S'(u_k))^{-1} S(u_k)$$

Méthode de Newton

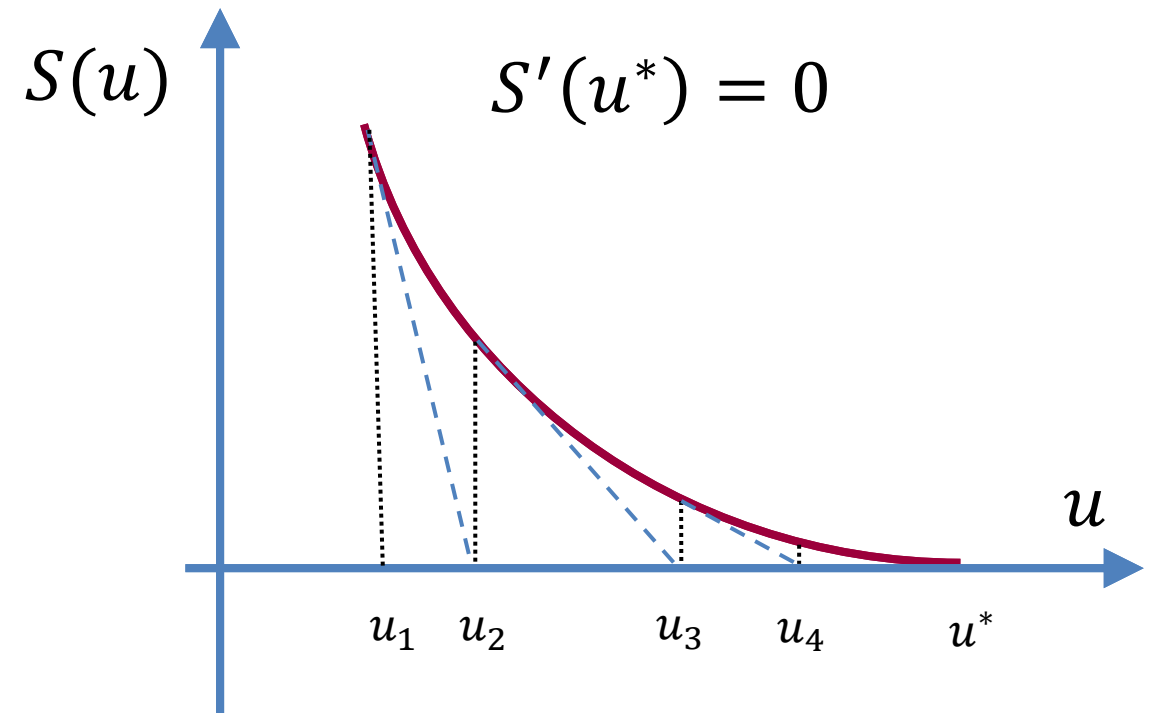
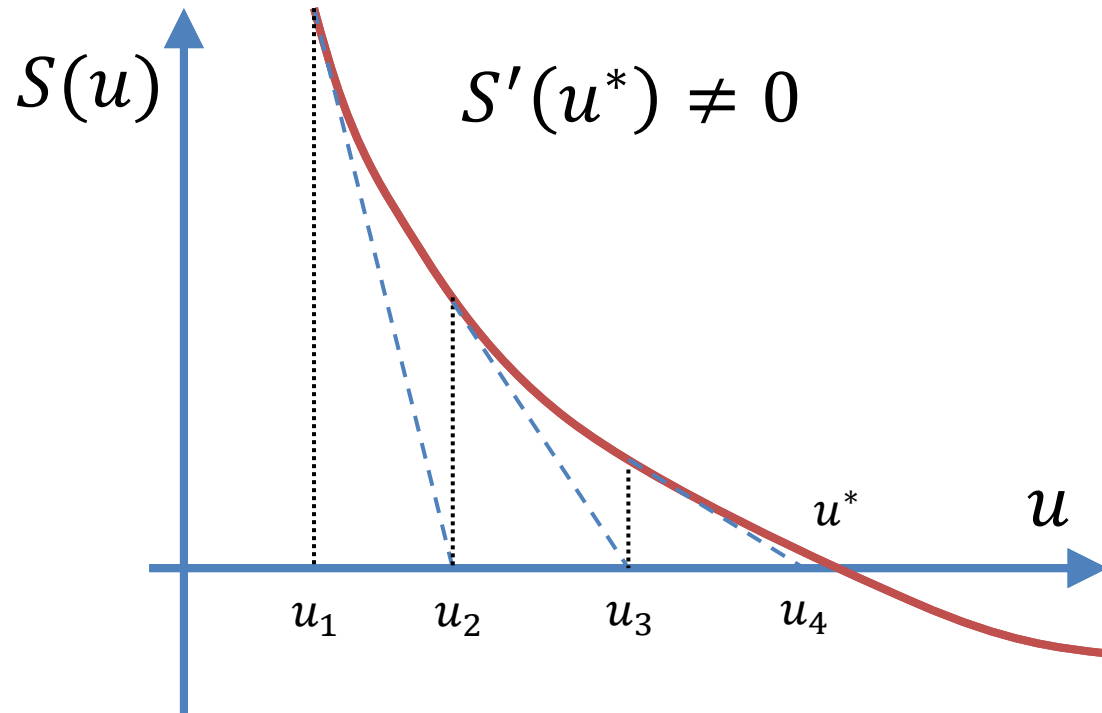
Limite

Question : Dans le cas scalaire, peut-on quand même appliquer la méthode de Newton si $S'(u^*) = 0$?



Méthode de Newton

Illustration graphique



- On peut bien appliquer la méthode de Newton si $S'(u^*) = 0$, mais on **perd la convergence superlinéaire**.

Exemple (en exercice) : $S(u) = (u - u^*)^m \Rightarrow \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} \rightarrow 1 - \frac{1}{m}$

Méthode de Newton

q-convergence

Supposons que u^* existe que S est dérivable et $S'(u^*) \neq 0$; on considère u_k proche de u^* . Formule de Taylor-Young sur $F: u \mapsto u - (S'(u))^{-1} S(u)$:

$$\begin{aligned} \exists \epsilon < \infty, \quad \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|^2} &\leq \frac{\|F(u^*) + F'(u^*)(u_k - u^*) - u^*\| + \epsilon \|u_k - u^*\|^2}{\|u_k - u^*\|^2} \\ &= \frac{\left\| \left(Id - [(S'(u^*))^{-1}]' S(u^*) - (S'(u^*))^{-1} S'(u^*) \right) (u_k - u^*) \right\| + \epsilon \|u_k - u^*\|^2}{\|u_k - u^*\|^2} = \epsilon \end{aligned}$$

La q-convergence de la méthode de Newton est **quadratique** (très rapide!)

Méthode de Newton

Convergence

Les conditions précédentes étant remplies (u^* existe, S dérivable et $(S'(u^*))^{-1} \neq 0$), la convergence est-elle assurée?



Méthode de Newton

Convergence

- La convergence LOCALE est assurée, mais certainement pas globale!

[Jupyter Notebook](#)

Méthode de Newton

Recherche linéaire

- Pour éviter de diverger, il faut introduire une **recherche de pas optimal** (recherche **linéaire**, dans une direction fixée):

$$u_{k+1} = u_k - \alpha (S'(u_k))^{-1} S(u_k), \quad \alpha \in (0,1]$$

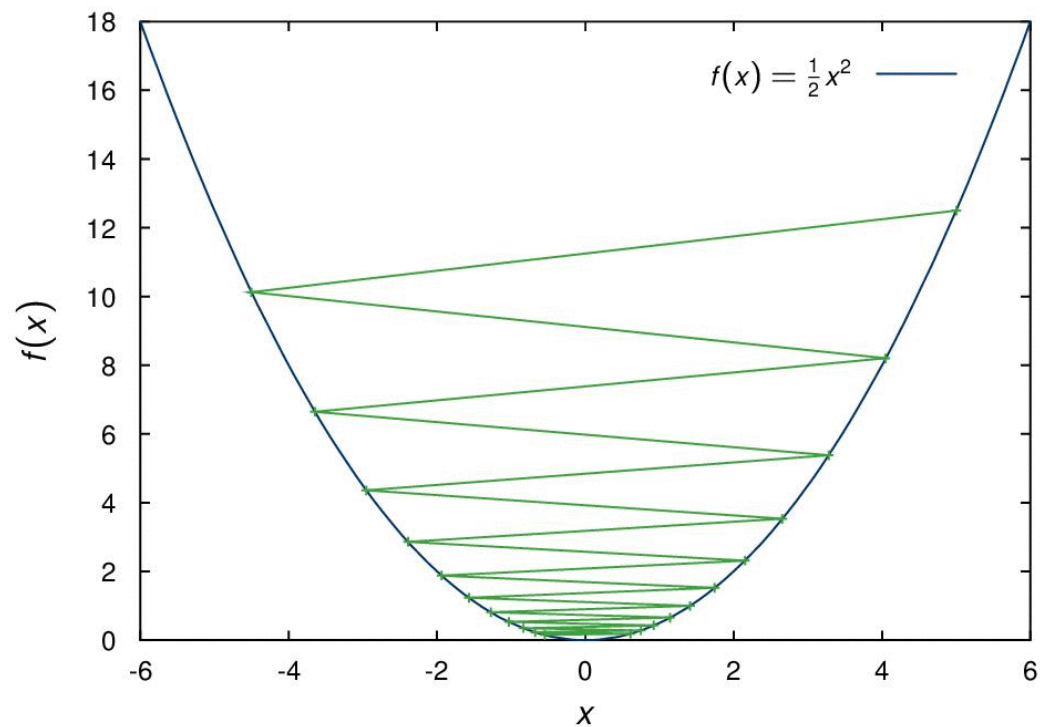
- En posant $f_k = S(u_k)^2$, on veut à chaque itération $f_{k+1} \leq f_k$
 $\Rightarrow S(u)$ se rapproche de 0
- La recherche linéaire fait sens si $d_k = -(S'(u_k))^{-1} S(u_k)$ est une **direction de descente** pour f_k . On peut le vérifier :

$$d_k \cdot f'_k = -2S(u_k)S'(u_k) (S'(u_k))^{-1} S(u_k) = -2S(u_k)^2 \leq 0$$

Méthode de Newton

Condition de décroissance (Armijo)

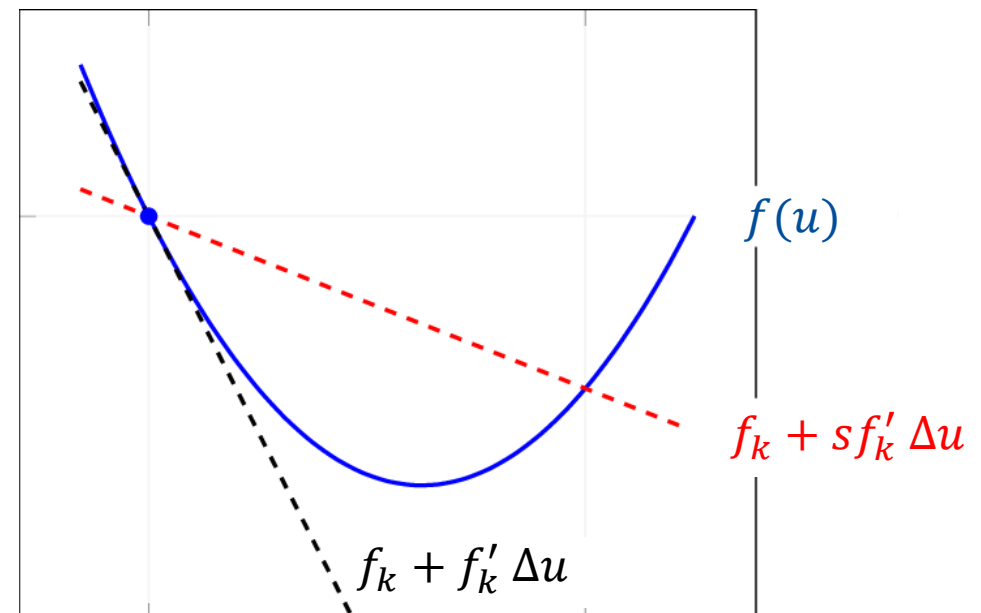
Le minimum attendu est que $f_{k+1} \leq f_k$, mais ce n'est pas forcément optimal



<https://medium.com/codex/gradient-descent-and-back-tracking-line-search-d8bd120bd625>

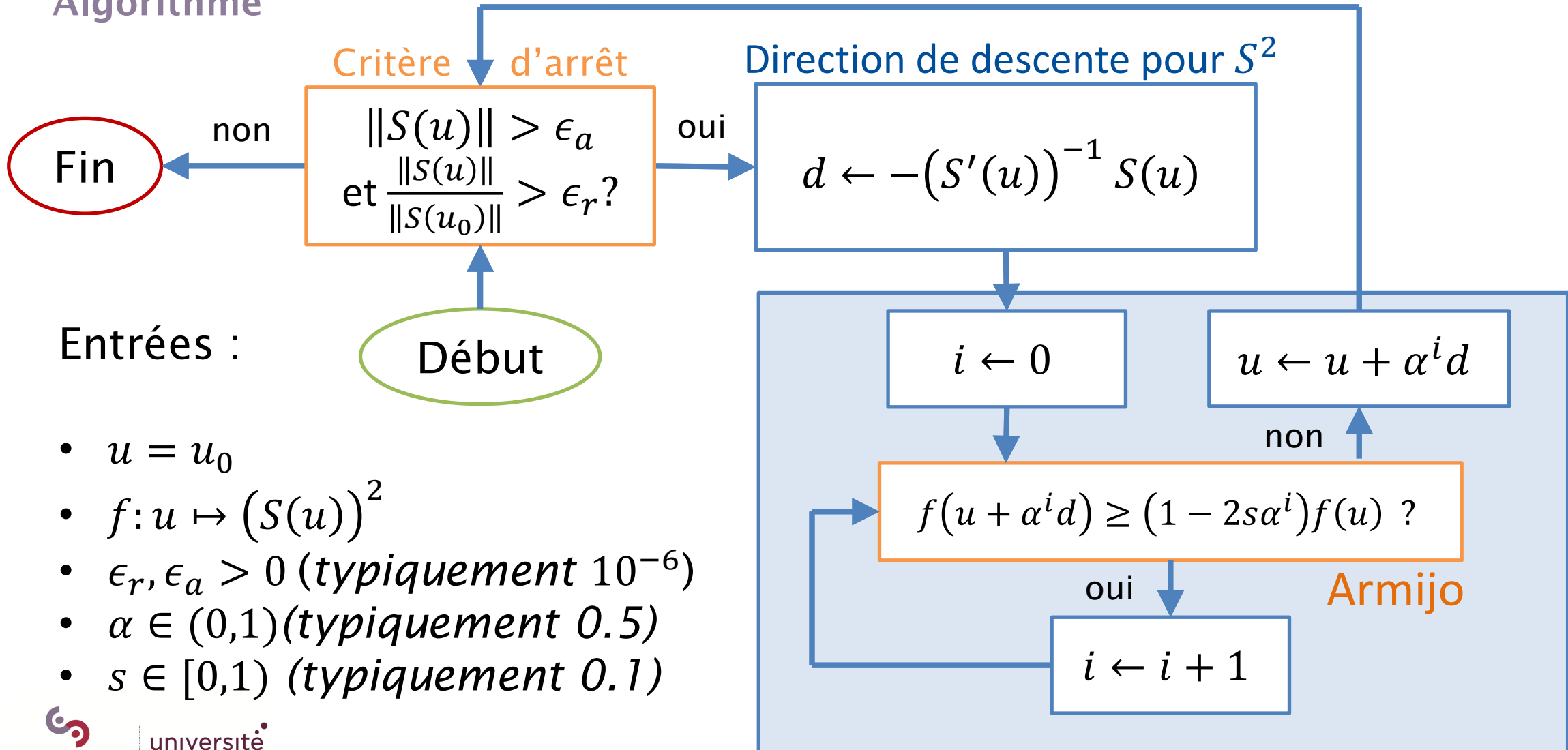
- On peut avoir des **oscillations** si le pas est trop grand. La **condition d'Armijo** permet d'éviter cela, avec $0 \leq s < 1$:

$$f_{k+1} < f_k + s f'_k (u_{k+1} - u_k)$$



Méthode de Newton

Algorithme



Entrées :

Début

- $u = u_0$
- $f: u \mapsto (S(u))^2$
- $\epsilon_r, \epsilon_a > 0$ (typiquement 10^{-6})
- $\alpha \in (0,1)$ (typiquement 0.5)
- $s \in [0,1)$ (typiquement 0.1)

Recherche linéaire
(backtracking)

Méthode de Newton

Application

[Livescript Matlab](#)

3) OPTIMISATION NON-CONTRAINT

Descente de gradient, méthode de Newton (bis)

Rappels

Problème direct

Jusqu'à présent, on a vu comment

- calculer l'état u d'un système (Point fixe, Newton)
- calculer un critère $C(u(x))$
- calculer sa dérivée $d_x C$ (différences finies, adjoint)

⇒ Résolution de **problèmes directs**

Généralement, ces problèmes sont **bien posés** (existence, unicité et stabilité de u)

Problème d'optimisation

Cas général

- L'optimisation se rapproche de la résolution d'un problème inverse.
- Il s'agit de trouver une **solution admissible** $x^* \in \Omega$, **minimisant un critère** en respectant des **contraintes**.

Un problème d'optimisation **mono-objectif** s'écrit sous forme standard

Trouver	$x^* = \arg \min_{x \in \Omega} F(x, p)$	Fonction objectif (coût)
Sous contrainte :	$h(x, p) = 0$	Contrainte d'égalité
	$g(x, p) \leq 0$	Contrainte d'inégalité

On se limite dans un premier temps aux **problèmes non-contraints**.

Descente de gradient – pas fixe

Principe

- **Propriété** : $-d_x F$ constitue une **direction de descente** de F

En effet, $f(x - \alpha d_x F(x)) = f(x) - \alpha (d_x F(x))^2 + o(\alpha) \leq f(x)$ pour $\alpha > 0$ suffisamment petit.

- On en déduit l'algorithme de **descente de gradient à pas fixe** :

$$x_{k+1} = x_k - \alpha d_x F(x_k)$$

Avec $\alpha > 0$ le pas.

Descente de gradient – pas fixe

Conditions d'arrêt

Quand l'algorithme s'arrête-t-il ?



Descente de gradient – pas fixe

Conditions d'arrêt

CN d'optimalité : $\|d_x F\| = 0$ (suffisante pour une fonction localement convexe)

⇒ Critère d'arrêt basé sur la norme de la dérivée : $\|d_x F\| < \epsilon_1$

Variantes :

- Norme infinie : $\max_{i=1,\dots,n} \frac{\partial f(x)}{\partial x_i} < \epsilon_2$
- Variation de la fonction objectif : $\left| \frac{f(x_{k+1}) - f(x_k)}{f(x_k)} \right| < \epsilon_3$
- Variation de la variable de conception : $\frac{\|x_{k+1} - x_k\|}{\|x_k\|} < \epsilon_4$

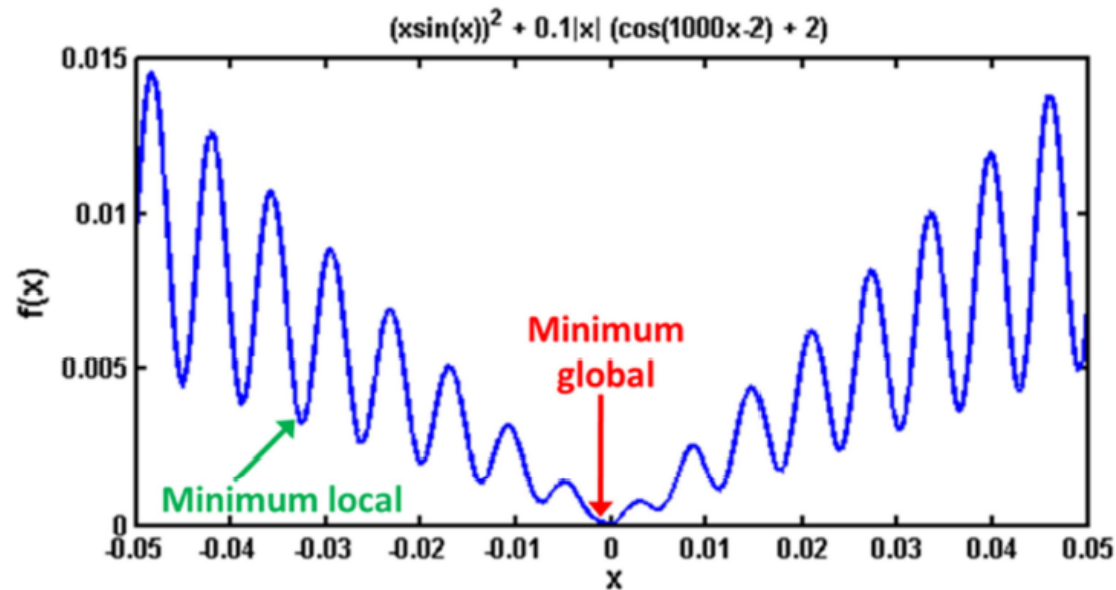
...

Remarque : toujours mettre un nombre d'itération maximal !

Optima locaux

Problème

- L'algorithme s'arrête quand $d_x F = 0$ (**point critique**, CN d'optimalité)
- Maximum local (instable), point selle (généralement pas un problème)
 - Autrement, il s'agit d'un **minimum local**...



Solutions possibles :

- Multi-start
- *Méthodes d'optimisation globale (cours P. Dessante).*

Dans la suite, on considèrera des fonctions fortement convexes :

$$F(tx + (1 - t)y) > tF(x) + (1 - t)F(y) \quad (\exists! \text{ minimum}).$$

Optima locaux

Exemples

Exemples d'optima locaux : topologies (libres) de machines électriques maximisant le couple moyen à alimentation fixée.



Résultats obtenus après initialisations aléatoires – (multi-start)

Descente de gradient – pas fixe

Convergence

- L'algorithme converge si le pas est suffisamment petit.
- Mais plus il est petit, plus il faut d'itérations

Illustration numérique

Livescript matlab

Descente de gradient – pas optimal

Principe

On résout à chaque itération:

$$\alpha^* = \arg \min_{\alpha} f(x_k - \alpha d_x F(x_k)) \quad [\text{recherche linéaire}]$$

$$x_{k+1} = x_k - \alpha^* d_x F(x_k) \quad [\text{mise à jour}]$$

- **Adaptation automatique du pas** : convergence **garantie** si le minimum existe et est unique.
- Pas forcément efficace (trouver le pas optimal peut être lent sans être nécessaire)
- On peut se contenter d'un *backtracking* par la méthode d'Armijo.

Illustration numérique

[Livescript matlab](#)

Descente de gradient préconditionnée

Principe

- **Préconditionnement** avec un opérateur positif P .

$$d_k = -P d_x F(x_k) \quad [\text{direction de descente}]$$

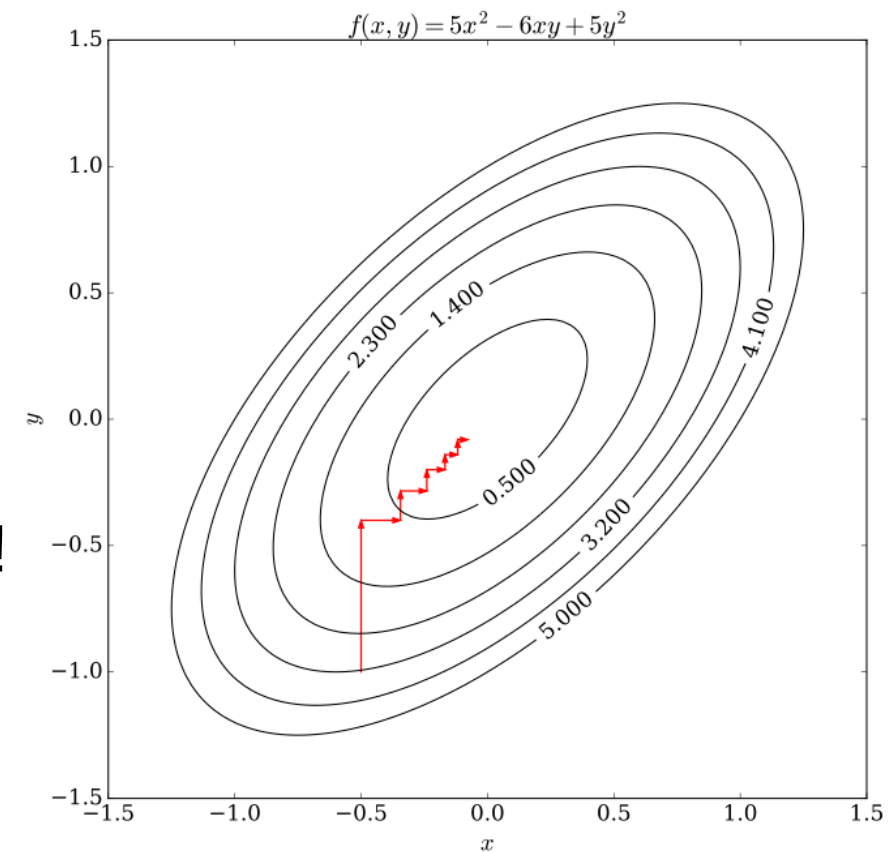
$$\alpha^* = \arg \min_{\alpha} f(x_k + \alpha d_k) \quad [\text{recherche linéaire}]$$

$$x_{k+1} = x_k + \alpha^* d_k \quad [\text{mise à jour}]$$

Exemple : « coordinate descent »

(*illustration numérique*), ou encore... Newton !

[Livescript matlab](#)



Méthode de Newton

Principe

- On a déjà vu le principe de la méthode de Newton pour chercher la racine d'une fonction.
- On peut donc appliquer cette méthode à $d_x F$ pour la recherche du **point critique** de F !

$$x_{k+1} = x_k - \alpha (d_{xx}F)^{-1} d_x F$$

- Ne fonctionne que si $d_{xx}F$ définie positive (caractérisation d'un minimum local, par rapport à un point selle ou un maximum local)
- Exact si la fonction est (localement) quadratique !
 - On peut décomposer en sous problèmes quadratiques : idée de SQP

Méthode de Newton

Application

- Banane de Rosenbrock

[Livescript matlab](#)

Méthode de Quasi-Newton

Principe rapide

- Le calcul de la dérivée première de F (« gradient » → **vecteur**) n'est pas toujours aisé (méthode de l'adjoint)
- Le calcul de la dérivée seconde (« hessien » → **matrice**) l'est encore moins ! Si x est de grande taille, on peut rencontrer des problèmes de mémoire.
- Méthode de la sécante, BFGS : on **réutilise les informations des étapes précédentes** pour estimer la matrice hessienne
- On peut quand même atteindre une convergence **superlinéaire**, même sans connaître la valeur exacte de la hessienne! Mais il faut s'assurer qu'elle soit **symétrique définie positive**.

Qu'avez-vous retenu de la séance ?

Mots-clés



OPTIMISATION SOUS CONTRAINTE

Reformulations, algorithmes

Problème d'optimisation

Cas général

- L'optimisation se rapproche de la résolution d'un problème inverse.
- Il s'agit de trouver une **solution admissible** $x^* \in \Omega$, **minimisant un critère** en respectant des **contraintes**.

Un problème d'optimisation **mono-objectif** s'écrit sous forme standard

Trouver	$x^* = \arg \min_{x \in \Omega} F(x, p)$	Fonction objectif (coût)
Sous contrainte :	$h(x, p) = 0$	Contrainte(s) d'égalité
	$g(x, p) \leq 0$	Contrainte(s) d'inégalité

Descente de gradient projetée

Principe

- Certaines contraintes sont **faciles** à prendre en compte.
- On peut donc projeter les variables sur l'espace admissible Ω .

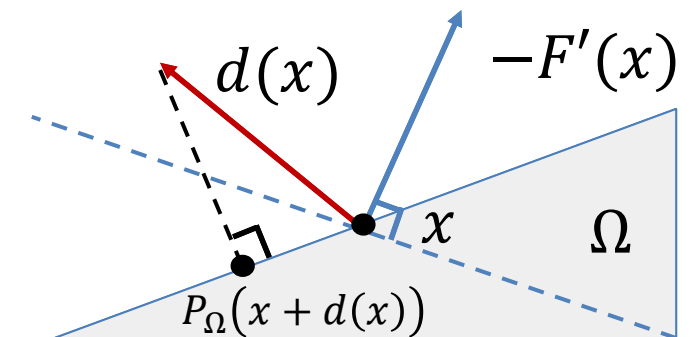
$$x_{k+1} = P_{\Omega}(x_k - \alpha F'(x))$$

Avec

- α le pas
- $F'(x)$ la dérivée de F
- P_{Ω} la projection orthogonale sur Ω (**pas toujours simple**). Dans le cas de contraintes de type « boîte » : $x > x_{min}$ ou $x < x_{max}$:

$$P_{\Omega}(x) = \max(x_{min}, \min(x_{max}, x))$$

Attention : **toute autre direction de descente ne l'est plus forcément après projection**



- **Critère d'arrêt** : nécessité de projeter aussi le gradient !

Pénalisation

Principe

- On peut transformer le problème contraint en un **problème non-contraint** (on peut alors appliquer les méthodes de descente).
- L'idée est d'ajouter à la fonction objectif à minimiser des termes de **pénalisation** importants lorsque les contraintes sont violées.

$$F_p(x) = F(x) + P_h(x) + P_g(x)$$

Avec

- $h(x) = 0 \Rightarrow P_h(x) = 0,$ $h(x) \neq 0 \Rightarrow P_h(x) \gg 1$
- $g(x) \leq 0 \Rightarrow P_g(x) = 0,$ $g(x) > 0 \Rightarrow P_g(x) \gg 1$

Pénalisation

Principe

Des idées de fonctions de pénalisation à utiliser ? (contrainte d'égalité)



Pénalisation

Principe

Des idées de fonctions de pénalisation à utiliser ? (contrainte d'inégalité)



Pénalisation

Extérieures

Exemple de pénalisation **extérieure** (C^1)

- Contrainte d'égalité

$$P_h(h(x)) = \frac{\mu_h}{2} h(x)^2$$

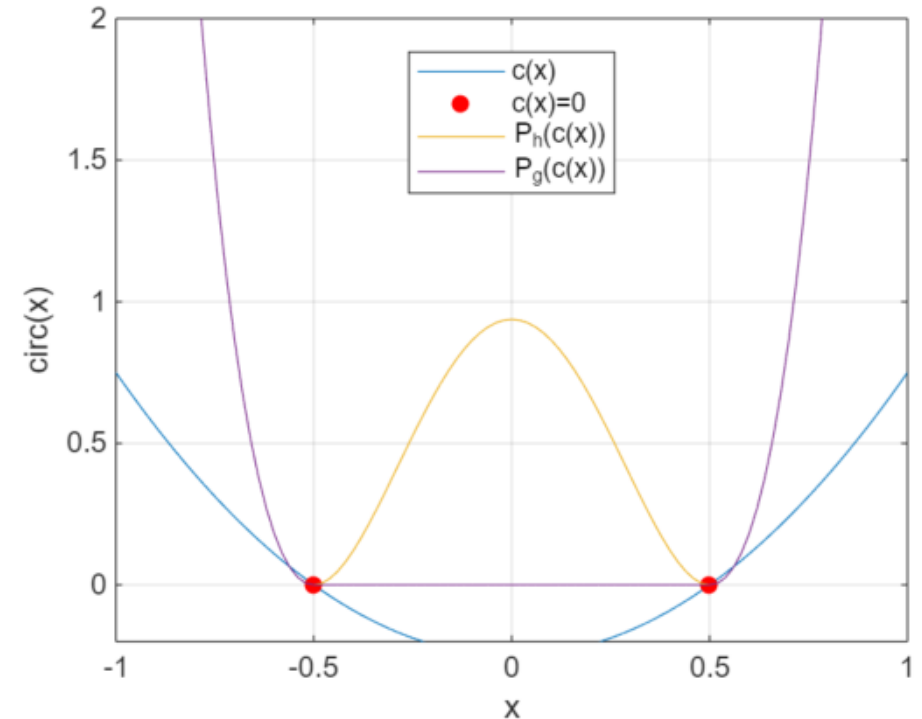
- Contrainte d'inégalité

$$P_g(g(x)) = \frac{\mu_g}{2} (\max\{0, g(x)\})^2$$

Si plusieurs contraintes :

$$P_h(x) = \sum_i \frac{\mu_i}{2} h_i(x)^2, \quad P_g = \sum_i \frac{\mu_g}{2} (\max\{0, g(x)\})^2$$

$$c(x) = x^2 - R^2$$



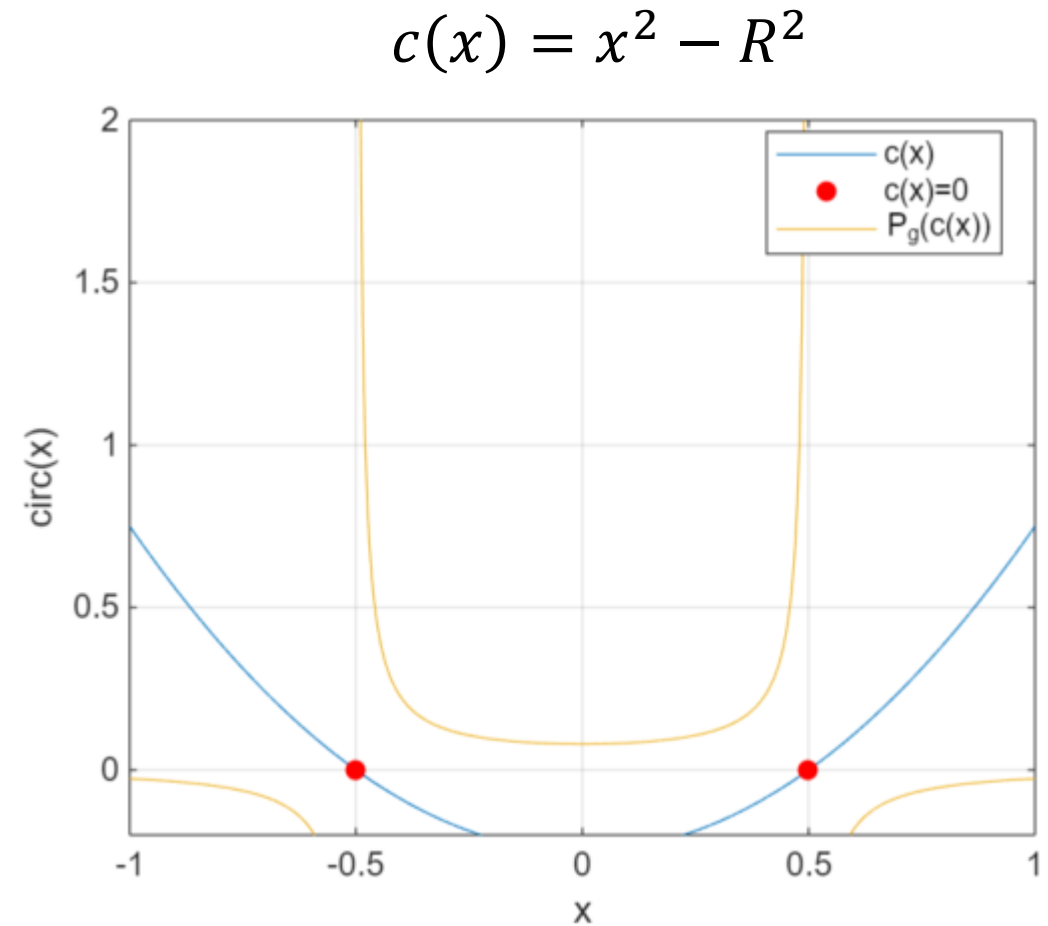
Pénalisation

Intérieure

Exemple de pénalisation intérieure (C^1)

- Contrainte d'égalité : **pas adapté !**
- Contrainte d'inégalité:

$$P_g(x) = -\mu_g \frac{1}{g(x)}$$



Pénalisation

Problèmes

- Les méthodes de pénalisation ne sont pas exactes lorsque μ_h et μ_g sont fixes.
- Si μ_h et μ_g sont variables pour converger exactement, alors on peut avoir un problème de **conditionnement**

Exemple numérique

Optimisation d'une inductance

Lagrangien

Problème dual

Rappelons le problème d'optimisation (**primal**)

Minimiser	$F(x)$	Fonction objectif (coût)
Sous contrainte	$h(x) = 0$	Contrainte(s) d'égalité
	$g(x) \leq 0$	Contrainte(s) d'inégalité

On définit le problème **dual** comme:

Maximiser	$\min_x F(x) + \lambda \cdot h(x) + \nu \cdot g(x)$	Lagrangien
Sous contrainte	$\nu \geq 0$	Contrainte(s) d'inégalité (boîte)

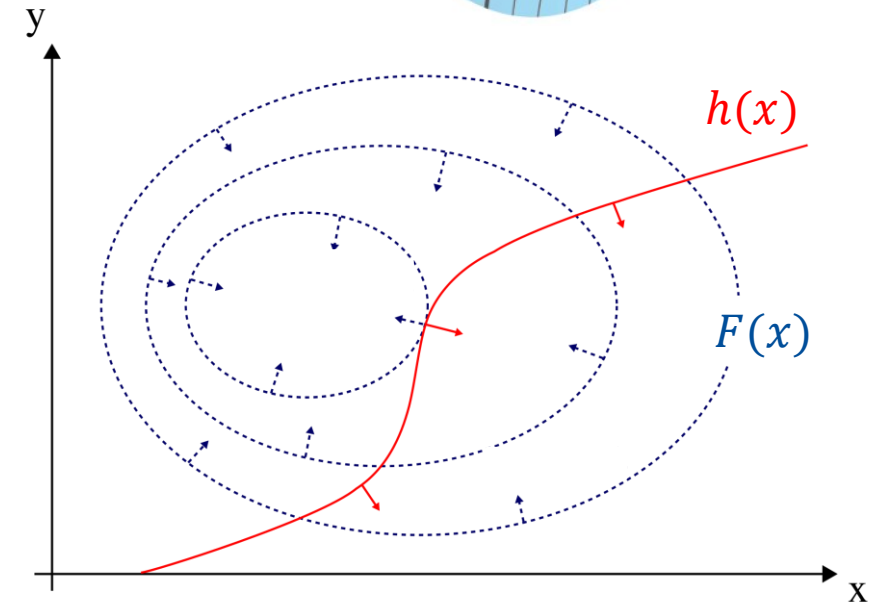
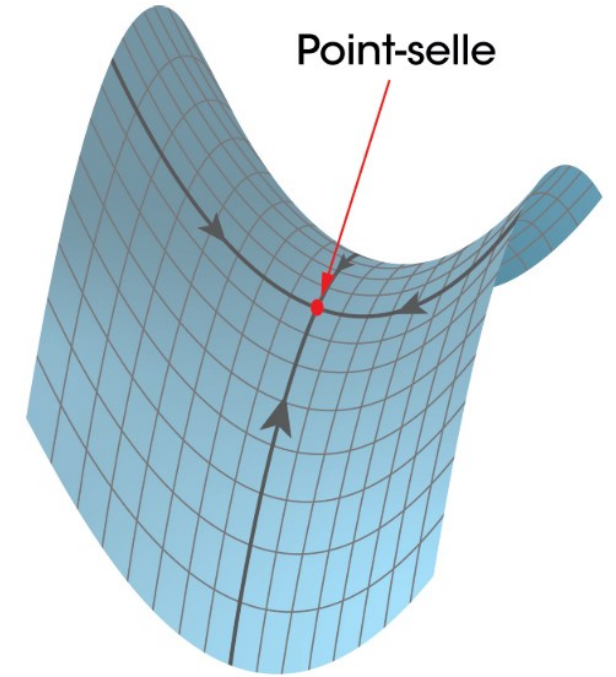
Lagrangien

Condition d'optimalité KKT

Si $\mathcal{L}(x, \lambda, \nu) = F(x) + \lambda \cdot h(x) + \nu \cdot g(x)$ admet un **point selle**, alors résoudre le problème dual est équivalent à résoudre le problème primal.

Conditions d'optimalité (Karush-Kuhn-Tucker)

- Stationnarité : $(\partial_x \mathcal{L}, \partial_\lambda \mathcal{L}, \partial_\nu \mathcal{L}) = (0, 0, 0)$
- Admissibilité (primale) : $h(x) = 0, g(x) \leq 0$
- Admissibilité (duale) : $\nu \geq 0$
- Complémentarité : $\nu \cdot g(x) = 0$



Lagrangien augmenté

Principe

- On considère des contraintes d'égalité (extension possible aux inégalités). On écrit le Lagrangien, et on ajoute un terme de pénalisation fixe

$$\mathcal{L}(x, \lambda) = F(x) + \lambda \cdot h(x) + \frac{\mu}{2} h(x)^2$$

On procède en **deux temps** (résolution du problème dual) :

1. Pour λ fixé, **minimisation de $\mathcal{L}(x, \lambda)$** par une méthode de descente
2. Une fois x^* trouvé, **mise à jour de λ pour maximiser $\mathcal{L}$** en fonction de la violation de la contrainte :

$$\lambda \leftarrow \lambda + \mu h(x^*)$$

Et on recommence jusqu'à convergence. Pas besoin que $\mu \rightarrow \infty$!

Principe

- Dans le lagrangien augmenté, on résout le problème « global » et « compliqué » à chaque fois.
- Une autre stratégie consiste à résoudre une succession de problèmes « locaux » et « simples » (car approchés).
- Par exemple, des **approximations quadratiques**, qu'on peut résoudre à l'aide de la **méthode de Newton**.
- Reste à vérifier les contraintes...

Principe

- On considère le problème d'optimisation quadratique **local**

Minimiser

$$F(x) = \frac{1}{2} x^T A x + b^T x$$

Sous contrainte $h(x) = 0$

Contrainte(s) d'égalité

- On pose le Lagrangien :

$$\mathcal{L}(x, \lambda) = F(x) + \lambda^T h(x)$$

Si $h = A_h x + b_h$, alors la condition de stationnarité $(\partial_x \mathcal{L}, \partial_\lambda \mathcal{L}) = (0, 0)$ s'écrit :

$$\begin{bmatrix} A & A_h \\ A_h^T & 0 \end{bmatrix} \begin{bmatrix} x^* \\ \lambda^* \end{bmatrix} = \begin{bmatrix} -b \\ -b_h \end{bmatrix}$$

- Et on peut résoudre directement. Si les contraintes ne sont pas linéaires, on peut appliquer la **méthode de Newton**.
- On recommence alors au voisinage du point suivant ; si les résultats sont conformes au modèle local, on étend la **région de confiance**.

CONCLUSION

Qu'avez-vous retenu de la séance ?

Mots-clés



Références

Cours, livres, sites internet

- Notes de cours, W. Zulehner, Special Numerical Methods: Nonlinear Equations
- Nocedal, J., & Wright, S. J. (2006). Numerical optimization. In P. Glynn & S. M. Robinson (Eds.), *Springer Series in Operations Research*. Springer.
<https://doi.org/10.1201/b19115-11>
- Ern, A., & Allaire, G., Optimisation et Contrôle,
<http://www.cmap.polytechnique.fr/~allaire/map435/poly435.pdf>
- Cerf, M. Techniques d'optimisation
<http://dumas.perso.math.cnrs.fr/MINT-2018-TO1.pdf>

Votre avis sur la séance

Evaluation et suggestions d'amélioration



<https://app.wooclap.com/IQMOEJ/questionnaires/67a07458d284be69f60fc4a8>

Exercices et problèmes

Livecoding

https://tcherrie.github.io/optim_ST7/lab/



Merci de votre attention

Méthodes d'optimisation à gradient

Théodore Cherrière, MCF CentraleSupélec, laboratoire GeePs
theodore.cherriere@centralesupelec.fr